OverlayCCL: Composing Collectives Above a Vendor Library via a Closed-Stack Search Loop

Anonymous Author(s)

Abstract

Picking the fastest collective-communication strategy for a distributed-training workload is hard. Timing each strategy on a microbenchmark is cheap but can pick a strategy that is slow or fails to compile inside a real training step. Running each strategy end-to-end is direct, but realistic GPU-cluster experiments are expensive. Cost-model simulators have been explored, but existing ones are tuned against a fixed reference environment, so their internal numbers drift across machines and model shapes. The simulator approach is even harder to get right on closed-source vendor stacks (AWS Trainium, Google TPU, Meta MTIA), because of two specific properties: *(C1) no visibility into the runtime* — when an operation is slow the user cannot inspect the vendor library to see why — and *(C2) no low-level control* — vendor primitives are called through a fixed API, with no exposed schedule-level intermediate representation (which would let a user choose how messages are routed inside a collective communication call).

We propose OverlayCCL, a closed-stack search loop that addresses each of these challenges. For (C1), an LLM agent builds the cost-model simulator on the target deployment itself: it writes probing code that measures the simulator's internal numbers against the actual device and model shape, so the simulator's estimates re-adapt to the deployment when either changes. For (C2), we accept the API as fixed and search one layer above it: which primitives to call, in what order, and how to reshape, pad, or slice the input/output tensors around each call. Against an internal-AWS-optimized baseline used in production on Trainium today, OverlayCCL delivers a **1.40× speedup** on OLMoE-10B at actual training runs — a 29% reduction in end-to-end steady-step time, with matched descent on real OpenWebText — and a **3.24× speedup** on Llama-7B. Per-primitive speedups hold across payload sizes on the AllToAllV bench and generalize to cluster sizes from 3 to 7 nodes. A multi-seed retest at $n \geq 5$ seeds yields clean $2\sigma_{\bar{x}}$ agent wins on every per-primitive (primitive, payload) cell, with loss floors bit-identical across seeds.

Keywords: collective communication, distributed training, LLM agents, AWS Trainium, cost-model simulator, closed-stack accelerators

1 Introduction

Choosing the fastest collective-communication strategy for a distributed-training workload is challenging, and two prevailing methods have substantial limitations. Running each candidate strategy end-to-end on the actual workload is direct but costly: realistic GPU-cluster experiments are expensive, and the cost grows with the number of strategies tried. Ranking strategies by a cheap microbenchmark on small isolated calls is much cheaper, but on a Trainium cluster we show that multiple strategies that win microbenchmarks are materially *slower* at actual training runs than a different strategy that loses in microbenchmarks (Figure 2). We call this *cross-scope inversion*: moving an operation from an isolated microbenchmark into a real multi-rank training step changes the per-call cost, because the surrounding training graph changes how the runtime launches, fuses, and lays out tensors for the call.

A third approach, scoring strategies against a cost-model simulator, has been explored in prior work (SimAI [61], ASTRA-sim [43, 63]). But existing simulators are written and tuned against a fixed reference configuration: the numbers they use internally — per-collective bandwidth, the overhead each time the runtime launches a graph, when adjacent calls fuse, etc. — drift across machines and model shapes, and re-tuning them by hand for each new environment or training task is again costly. The simulator-based approach gets even harder on the closed-source vendor stacks production training increasingly runs on (AWS Trainium, Google TPU, Meta MTIA), because of two specific properties of these stacks. **(C1) No visibility into the runtime:** when an operation (e.g., `xm.all_to_all` from Trainium) is unexpectedly slow, developers cannot inspect the vendor library to see why — its internals are not source-available, and there are limited per-primitive configuration parameters to calibrate a simulator against. **(C2) No low-level control:** the vendor's collective primitives (`xm.all_gather`, `xm.all_to_all`, ...) are called through a fixed API; there is no exposed schedule-level intermediate representation — which, in the open-stack systems MSCCL [12] and TACCL [50] target, is the surface that lets a user choose how chunks are split, routed through the network, or fused inside a collective.

This paper introduces a new design that addresses each of these challenges. **For (C1)**, we approximate visibility into the closed runtime by having an LLM agent build a cost-model simulator on the target deployment itself: the LLM

writes probing code that measures the simulator's internal numbers against the actual device and model shape the workload will use. This does not require internal visibility into the runtime, but it gives a usable proxy for "is this strategy fast for this specific task?" whose estimates re-adapt to the deployment when either the device or the model shape changes — so the simulator stays workload-faithful where a hand-coded one would drift. **For (C2)**, we do not try to recover low-level control. We accept the vendor API as fixed and search one layer above it — over which primitives to call, in what order, and how to reshape, pad, or slice the input/output tensors around each call. The LLM proposes strategies at this layer; the workload-calibrated simulator scores them; top-scoring strategies are validated on real hardware before deployment.

One might ask: if the simulator's own internal numbers come from on-device probes, isn't this just the microbenchmark option in a different wrapper? Our ablation (§4.2) is set up to answer exactly that. The difference is in *what* we probe and *how* we probe it. Our simulator encodes hardware-specific performance models, and the LLM is guided to calibrate them using probe workloads that resemble real model-training settings (real tensor shapes, full step boundaries, multi-rank dispatch patterns), rather than the small isolated calls a standalone microbenchmark would measure. The probes therefore capture behavior that small microbenchmarks miss, while remaining far cheaper than running a full training harness end-to-end on each candidate strategy. When we ablate the simulator and let the LLM rank strategies by its own small microbenchmarks instead, the 1.40× OLMoE-10B speedup collapses to 1.00×: the LLM converges to structurally the same strategy practitioners already deploy, because microbench rankings are exactly what selected that strategy in the first place. In other words, the work-load-faithful simulator, not the LLM, is what picks the strategy that wins at actual training runs.

Closed-stack collective communication library. A growing class of production training accelerators operates on proprietary collective communication libraries. AWS Trainium [1, 2] — 30–40% better price-performance than NVIDIA H100 on transformer workloads and adopted at scale for Mixture-of-Expert (MoE) and Llama training — ships a Neuron Runtime that exposes collectives only through the PyTorch/XLA integration (XLA = Accelerated Linear Algebra, PyTorch's compiler bridge to non-NVIDIA accelerators) — i.e. as `xm.*` entry points such as `xm.all_gather` — behind which the Neuron compiler emits NEFF binaries (Neuron Executable File Format; the compiled artifact loaded onto each NeuronCore) from unpublished sources. There is no user-configurable schedule layer: per-collective chunking and network-interface-card (NIC) routing cannot be reached from above the runtime. Google TPU and Meta MTIA exhibit the same constraint: the programmable schedule surface that MSCCL- and TACCL-style synthesis depend on is not exposed. The schedule-synthesis paradigm of the open-stack era cannot be ported here in any straightforward way. Above the `xm.*` (or equivalent vendor) boundary, the design choice that remains is which primitive to call, in what order, and against what tensor layout. We refer to this as the *strategy* layer, in contrast to the *schedule* layer that MSCCL/TACCL operate on, which decides how each collective message is split into chunks, which network paths each chunk takes, and how the chunks are pipelined across devices. The strategy layer is less expressive but, as we show, contains strategies 1.4–3.2× faster than the developer baseline.

Related schedule-synthesis work and why it does not apply here. Automated synthesis of collective-communication algorithms has been studied at the *schedule* layer — which message chunks move along which paths, in what order, against which network adapters. MSCCL and MSCCLang [12] emit XML or DSL schedules for the NCCL/RCCL runtime [39] to consume; TACCL [50] uses a sketch language and an SMT solver; AutoCCL [64] adds tuning; SCCL [8] synthesizes Pareto-optimal algorithms from first principles. All share an infrastructural prerequisite: the underlying runtime exposes a programmable intermediate representation (IR) for the schedule.

Validation and contributions. We validate our design on the eight collective problems listed in Table 1 — four that arise in Mixture-of-Expert (MoE) training [52] and four that arise in dense-transformer (Llama-style) training — on a 7-node `trn1.32xlarge` cluster (224 ranks).

Problem	Role in the training step
MoE-side	
AllToAllV [29]	MoE token dispatch/combine; counts vary
Uniform AllToAll [69]	MoE dispatch/combine; counts equal
Ring KV [32]	rotate KV shards around the rank ring
Distributed CE [54]	loss over rank-sharded vocabulary
Llama-side	
PP cross-stage [19]	activations/grads between PP stages
TP MLP [54]	all-reduce after MLP in tensor parallel
FSDP weight prefetch [67]	gather sharded weights per layer
Layer-block AR [26]	all-reduce grads at layer blocks

Table 1. The eight collective problems we evaluate.

The baseline we measure against is the *internal-AWS-optimized strategy* AWS uses in production today to train large-scale MoE and dense transformer models on Trainium, built and maintained by five experienced AWS developers. On OLMoE-10B the deployed agent strategy delivers **1.40**× steady-state step-time speedup with matched descent on a 2500-step real-OpenWebText [17] AdamW run; on Llama-7B it delivers **3.24**×. Per-primitive per-step speedups range 4–12×, above the end-to-end ratio because constant non-collective compute dilutes the headline.

Deployment cost. A 1.40× speedup on OLMoE-10B is worth roughly the following in a typical training deployment. A single 7-node `trn1.32xlarge` cluster runs at ~$19/hour on-demand; a 14-day pretraining job at that scale is ~$47k in compute alone. A 1.40× end-to-end speedup on the steady-state step time cuts that to ~$34k — ~$13k saved per pretraining, or ~29% of the compute budget — with the same descent trajectory. The one-time cost of running the search is ~$19 in Sonnet 4.5 LLM tokens and ~56 min of the 7-node cluster (Table 10). At the scale of production MoE and Llama training the search cost amortizes across a single pretraining and every fine-tune that reuses the deployed runtime module. The Llama-7B 3.24× number sits at the top of this range because per-microbatch dispatch overhead (M dispatches per step in the baseline, one in the bundled agent) is the ratio-limiting term.

The contributions are (i) to our knowledge the first system that finds faster collective-communication strategies on top of a closed-source vendor library, delivering **1.40**× end-to-end on OLMoE-10B and **3.24**× on Llama-7B over the production baseline; (ii) a cost-model simulator whose per-term constants are re-fit per-cluster and per-model-shape by LLM-written on-device probes, keeping the simulator workload-faithful without hand-coding new probe scripts for each deployment; and (iii) the closed-stack search loop combining the above with LLM-driven mutation and hardware validation, with code emitted as drop-in runtime modules and algorithm-equivalent descent verified on real tokens.

Scope and generalization. We instantiate our design on AWS Trainium. The six cost-model terms are Trainium-specific, but the categories they fall into (per-graph-launch overhead, per-collective bandwidth ceiling, device-vs.-host copy cost, runtime cache-reload cost) are general to closed-source vendor runtimes that compile their own collective binaries. Google TPU and Meta MTIA each expose closed equivalents; we expect the strategy-search design to port with the cost-model term set re-calibrated, and leave that to follow-up work.

2 System Model

2.1 The search loop

Notation. A collective problem p has a pure-Python reference $\text{ref}_p(\cdot)$ encoding correctness; a seed library $\mathcal{S}_p$ of human-written candidate strategies; and fixed I/O shapes. The search space Σ_p is the set of functions with p's signature that compose primitives from the vendor set

$$\mathcal{V} = \{\texttt{all\_gather}, \texttt{reduce\_scatter}, \texttt{all\_reduce}, \texttt{collective\_permute}, \texttt{all\_to\_all}, \ldots\}$$

together with local host-side ops the accelerator's compiler lowers to device code. A candidate $s \in \Sigma_p$ is *admissible* if it matches the reference at multiple world sizes —

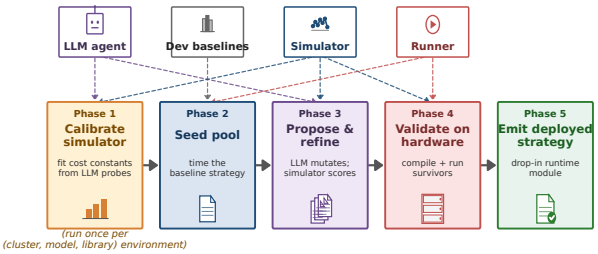

Figure 1. Closed-stack search loop. The training workload supplies tensor shapes and the primitive set. An LLM agent plays two roles: it writes hardware-specific probing code that calibrates a workload-faithful cost model on the target device, and it proposes candidate strategies whose predicted step time the cost model scores. The top-scoring candidate is validated on real hardware before being emitted as a drop-in runtime module. The rest of this section expands the loop into five sequential phases.

$\textsc{Correct}(s) = 1$ iff $s(\cdot) = \text{ref}_p(\cdot)$ for every $\text{ws} \in \{4, 8\}$ — and passes a hardware gate at the training-shape configuration ($\textsc{Gate}(s) = 1$ iff the strategy compiles and runs without error). Phase 1 fits a cost model $\text{Sim} : \Sigma_p \to \mathbb{R}_{\geq 0}$ from on-device measurements (Section 2.2); Phase 4 evaluates Gate on the target device. The deployed strategy is $s_p^\star = \arg\min_{s \in \Sigma_p^\dagger} \text{Sim}(s)$ over $\Sigma_p^\dagger = \{s \in \Sigma_p : \textsc{Correct}(s) \wedge \textsc{Gate}(s) = 1\}$. The LLM $\mathcal{M}$ acts as a stochastic proposer $\mathcal{M} : \Sigma_p \to \Sigma_p$ that mutates candidates; Phase 3 instantiates this as one of three search shapes (strategy-enumerate / cc-react / multi-island) over a bounded $\mathcal{M}$-call budget. The five sequential phases that implement this loop are illustrated in Figure 1.

Phase 1: agent-driven on-device calibration. Throughout the paper we refer to the LLM-driven search loop as the *agent*: a process that proposes, scores, and refines candidate strategies through several phases. In Phase 1, the agent is given a set of measurement *tools* (listed in Table 2) and designs the probe campaign autonomously: which tensor sizes to sweep, which primitives to compare, how many repetitions to run, and when to test whether a measured number really comes from the term it is meant to isolate (e.g. rule out hidden memory-copy overhead by varying whether the input tensor is stored contiguously in memory). Each tool returns an empirical scalar or function that wires into one term of Eq. 1; the agent produces the numerical constants that populate the simulator. We supply the tools (so the LLM cannot fabricate numbers), not the experimental design (so the LLM exploits domain knowledge to choose more informative probes than a hardcoded script would). Two terms used in the

tool list are worth defining: *EFA* (Elastic Fabric Adapter) is AWS's RDMA-based cross-node network fabric, used by the cross-node point-to-point probe; *amortization*, in the back-to-back tool, refers to the empirical observation that the cost of k consecutive collective calls is typically less than k times the cost of one call, because the runtime can overlap some of the per-call overhead across consecutive calls.

Tool	What it measures
`m_collective_lat`	per-prim latency vs. bytes / world size
`m_p2p_transfer`	cross-node point-to-point bandwidth over EFA
`m_xla_op_overhead`	per-op floors for local XLA ops
`m_compilation_cost`	cold / cache / reload NEFF compile times
`m_launch_overhead`	per-graph-dispatch (`mark_step`) tax
`m_memcopy_thru`	seq. and strided host/device copy bw
`m_b2b_amortization`	amortization curve for k consecutive collectives

Table 2. Phase-1 measurement tools the agent is given (prefix `m_` = `measure_`).

Phase 2: Baseline evaluation. A per-problem library of seeded templates is scored through the simulator. The seed library always contains the *internal-AWS-optimized baseline* strategy for that primitive plus a small number of workable but typically naive or slower alternatives. The baseline we measure against throughout the paper is the strategy AWS *uses in production today* to train large-scale MoE and dense transformer models on Trainium. It is built and confirmed by five experienced AWS developers who write and maintain Trainium training paths professionally. The result is the optimized strategy practitioners actually deploy, not a strawman. The LLM is not told which seed is the baseline pick. This both calibrates the simulator's ordering against established practice and seeds Phase 3.

Phase 3: Agent-driven search. The search style we use is **strategy-enumerate**: the LLM enumerates K structurally distinct strategies as natural-language sketches ("pack and gather", "per-source slice loop", "masked all-reduce over stacked microbatches", ...), implements each independently, then refines the top-2 by simulator score under a bounded budget (§2.4).

Phase 4: Hardware validation. Each Phase-3 survivor runs through a 20-iteration hardware (HW) microbench plus a 10-step training-validation (TV) harness on a synthetic 8-layer LM. Crashes, OOMs, NaN at world-scale, or shape-gate failures exclude the candidate regardless of simulator score; failed candidates get LLM-assisted recovery (up to two attempts). HW timing itself is not the ranking signal.

Phase 5: Code generation. The lowest *simulator-score* survivor among HW+TV passers wins. We do not pick by HW microbench directly because the microbench measures isolated-call latency with cached NEFFs and does not surface the NEFF cache-eviction cost that appears in real training.

Algorithm 1 End-to-end closed-stack collective-strategy search.

Require: P, ref$(\cdot)$, $\mathcal{S}_P$, LLM $\mathcal{M}$, profiling tools $\mathcal{T}$, hardware H.
Ensure: deployable runtime module for problem P.

$\Theta \leftarrow \mathcal{M}.\textsc{calibrate}(\mathcal{T}, H)$ ▷ Phase 1
$\mathrm{Sim}(\cdot) \leftarrow \textsc{BuildCost}(\Theta)$ ▷ Eq. 1
for $s \in \mathcal{S}_P$ **do** ▷ Phase 2
  $\mathrm{score}(s) \leftarrow \mathrm{Sim}(s)$; $\textsc{Verify}(s, \mathrm{ref}, \mathrm{ws}{\in}\{4, 8\})$ ▷ verify on small rank counts $\mathrm{ws} \in \{4, 8\}$
end for
$C \leftarrow \textsc{InitPool}(\mathcal{S}_P)$ ▷ Phase 3 (strategy-enumerate)
for $r = 1 \ldots R$ **do**
  $s' \leftarrow \mathcal{M}.\textsc{propose}(C, \Theta)$
  if $\neg\, \textsc{Verify}(s', \mathrm{ref})$ **then continue**
  end if
  $\mathrm{score}(s') \leftarrow \mathrm{Sim}(s')$; $C \leftarrow C \cup \{s'\}$.
end for
for each Phase-3 survivor s **do** ▷ Phase 4
  $t_{\mathrm{hw}}(s) \leftarrow \textsc{Microbench}(s, H)$
  $\textsc{ShapeGate}(s, H)$; $\textsc{TVGate}(s, H)$.
  $s \leftarrow \mathcal{M}.\textsc{recover}(s)$ **if** gate fails (up to 2×).
end for
$s^\star \leftarrow \arg\min_{s \in \mathrm{survived}} \mathrm{Sim}(s)$ ▷ Phase 5
$\textsc{Emit}$(runtime module for P, $s^\star$); deploy to H.

2.2 Cost model

The simulator builds a per-step time estimate as the sum of six physically-grounded terms:

$$T_{\mathrm{step}} = \underbrace{\textstyle\sum_{i \in \mathcal{L}} c(\mathrm{op}_i, b_i)}_{T_{\mathrm{local}}} + \underbrace{\textstyle\sum_j \max(d^{(j)}_{\mathrm{amort}}, b_j / w_{\mathrm{eff}})}_{T_{\mathrm{bw}}} + \underbrace{d_{\mathrm{full}} + (k{-}1)\, d_{\mathrm{amort}}}_{T_{\mathrm{coll}}} + \underbrace{2\,\ell\, m}_{T_{\mathrm{launch}}} + \underbrace{C(b_{\max})\, r/S}_{T_{\mathrm{NEFF}}} + \underbrace{B/w + \pi_{\mathrm{net}}}_{T_{\mathrm{net}}}. \quad (1)$$

2.2.1 Per-op local cost (T_{local}). T_{local} sums per-op cost over $\mathcal{L}$, the chronological list of local XLA ops the simulator records at trace time. View-only and fused-elementwise ops pay a small fixed floor; concatenation and stacking (`cat`/`stack`) are bytes-aware, with cost $\max(\mathrm{floor}, b/w_{\mathrm{seq}})$ where b is bytes moved and w_{seq} is the sequential memory bandwidth measured in Phase 1; contiguity-dependent ops like `reshape`/`contiguous` pay $\max(\mathrm{floor}, b/w_{\mathrm{strided}})$ on non-contiguous sources, using the

lower strided bandwidth measured for non-unit-stride patterns. This blocks the agent from winning with a chain of `narrow→reshape→contiguous` that looks cheap in isolation but moves the gathered buffer at strided bandwidth. `index_select` and host-side `torch.tensor(...)` are volume-scaled by the same rule. All remaining ops pay their measured isolated cost. Op-category floors and the `TrackedTensor` dunder-recording rules follow the same shape.

2.2.2 Collective bandwidth and back-to-back amortization (T_{bw}, T_{coll}).

T_{bw} floors each dispatch at b_j/w_{eff}, where w_{eff} comes from the Phase-1 topology object: cross-node, w_{eff} is the per-EFA-adapter bandwidth times the number of adapters per node (Elastic Fabric Adapter, AWS's high-bandwidth NIC); intra-node it is the per-NeuronLink bandwidth (NeuronLink is Trainium's on-package interconnect between NeuronCores). Nothing is hardcoded per problem. T_{coll} charges k back-to-back dispatches with an amortization schedule fit empirically: *back-to-back amortization* is the empirically observed drop in per-call cost when several collectives fire consecutively inside one launched graph, because the launch tax and pipeline-fill cost are paid only once. A shallow chain ($k = 1$) pays the full per-call cost; a moderate chain (a few back-to-back calls) pays a discounted rate as the network pipeline fills; a deeper chain pays an even lower rate because the XLA compiler can fuse adjacent collectives into one NEFF segment. The discount factors are auto-fit from `measure_back_to_back_amortization` probes; the fitting procedure, converged values on our cluster, and the run-reset rules that prevent `*inv`-interleaving from claiming the same amortization follow the same fit.

2.2.3 Graph-launch and NEFF reload (T_{launch}, T_{NEFF}, T_{net}).

T_{launch} pays ℓ from `measure_graph_launch_overhead` twice per `autograd.Function` boundary (forward `mark_step` + implicit backward `mark_step`). The simulator AST-derives an outer-loop tax that contributes to m in this term; inner per-tensor/per-bucket loops do not pay it because XLA fuses them into one NEFF chain. T_{NEFF} amortizes per-NEFF reload cost $C(b_{\max})$ over S steps with r reload events under the default small-cache configuration; T_{net} adds the remaining static-byte transfer term. The exact AST rules for identifying microbatch loops and the per-cluster `per_graph_neff_load_us` default follow the same measurement schedule.

2.2.4 Reward-hacking-resistant structural terms.

Three structural terms catch HW failure modes that pure-cost ranking misses.

1. A **fusion-credit** pass discounts each local compute op adjacent to a collective with no `stack`/`cat` barrier between them, capturing the compiler-level fusion of element-wise/reduction ops into a collective's segment.
2. An **HBM safety** term guards against running out of high-bandwidth memory (HBM is the per-NeuronCore on-package memory budget, 16 GB on `trn1`). Rather than hard-rejecting any candidate whose peak memory exceeds the budget — which would let a single conservative estimate disqualify an otherwise good strategy — we apply a smooth quadratic penalty proportional to how far the estimated peak overruns the budget. A separate abstract-syntax-tree (AST) pass — AST is the parsed structure of the candidate's source code — recognizes explicit byte-budget assignments (e.g. a `chunk_size_bytes = ...` hyperparameter that bounds a bucketed all-reduce) and propagates that cap into scaled-byte estimates, so a bucketed algorithm is not charged for the much-larger flat tensor it would otherwise appear to materialize.
3. **Primitive-viability** terms encode two real HW failure modes: `xm.collective_permute` rings at large world sizes (compiler aborts) and `xm.all_reduce` payloads above the per-collective size limit both receive $+\infty$ cost.

Calibration constants, regex lists, and the world-size failure pattern are Phase-1 outputs auto-fit per cluster.

2.2.5 Phase-5 ranking: why simulator score, not raw microbench.

Phase 5 ranks survivors by simulator score, not raw HW microbench latency or TV step time. The HW microbench and the TV harness are correctness gates: they catch crashes, NaNs, and shape failures but do not drive ranking. There are two reasons. First, HW and TV measurements often *fail to generalize to real multi-rank training*: the microbench runs at small world size with cached NEFFs and warm caches, the TV harness runs a tiny 8-layer LM that does not exercise the per-step NEFF reload and back-to-back amortization patterns that dominate at training shape and 224 ranks — a strategy that wins at small-scope wall clock may lose at training scope, and vice versa (this is the cross-scope-inversion finding the paper documents). Second, a raw wall-clock number is a sparse reward (no per-op attribution) that an LLM can game — stacking metadata-only views that fuse for free in isolation but force an extra `mark_step` in real training, or moving host-side index construction into a discarded warmup phase. The simulator decomposes the reward into per-term contributions with tool-measured floors, so an optimizer that pushes one term too low while leaving the others unchanged stops getting credit. The gate fallback rule is: when no Phase-3 candidate clears HW+TV gates, deploy the lowest-simulator-score baseline rather than a sim-only winner that cannot load.

2.3 On-device profiling (Phase 1): the agent builds its own simulator

The cost model of Section 2.2 is not a fitted regression nor an offline lookup table: each of its parameters $\Theta = (\Theta_{\text{net}}, \Theta_{\text{launch}}, \Theta_{\text{NEFF}}, \Theta_{\text{copy}}, \Theta_{\text{neff-cache}})$ is produced by the agent itself, which calls the probe tools listed in Table 2 on the deployment hardware. In plain terms: the network parameter Θ_{net} is the straight-line cost model $T(b,N)=\alpha(N)+\beta(N)\,b$ for each collective primitive at N ranks and b bytes, fit from `m_collective_lat`; Θ_{launch} is the fixed per-graph-launch overhead measured by `m_launch_overhead` (about 50–200 μs on Trainium); Θ_{NEFF} summarizes how long it takes to compile and load a NEFF, measured by `m_compilation_cost` at four cache tiers (cold compile, in-process cache, disk NEFF cache, warm re-use) and amortized across the typical number of steps a NEFF stays resident ($K_{\text{amort}}\approx$5000 on our setup); Θ_{copy} is the sequential and strided memcopy bandwidth from `m_memcopy_thru`; and $\Theta_{\text{neff-cache}}$ records the back-to-back collective amortization curve from `m_b2b_amortization` at varying queue depth. The exact tool API and sweep grids track the primitives listed in Table 2.

Phase 1 calibration takes 8–12 wall-minutes on the 7-node cluster, runs once per (hardware, software-version) tuple, and drops its measurements into Eq. 1 verbatim. This is load-bearing: a closed vendor stack updates its collective implementation, runtime scheduler, and NEFF format across versions, and the only honest way to track that drift is to re-measure Θ against the installed stack rather than extrapolate from a vendor whitepaper. The ablation in Section 4.2 confirms it: hiding Θ from the in-loop scoring collapses OLMoE-10B end-to-end from 1.40× to 1.00× because the LLM-judge falls back to whichever pattern wins a per-call microbench at small shapes — structurally the inline developer baseline. Turning 7-node microbench per-call latency into a faithful training-step prediction is what lets the search find strategies whose per-call latency loses but whose per-step contribution wins.

2.4 Strategy-enumerate loop (Phase 3)

Phase 3 takes the Phase-2 seed pool $\mathcal{S}_P = \{s_0^{(1)}, s_0^{(2)}, \ldots\}$ (the SOTA developer strategy plus naive or slower alternatives, each scored by $\text{Sim}(\cdot)$) and runs a bounded LLM-evolution loop on top. We use an LLM as the mutation oracle here, rather than classical search methods (e.g. random sampling, simulated annealing, or a genetic algorithm that mutates a fixed grammar of code templates) because the task is to *author* candidate code that will be deployed, not to pick a point on a fixed grammar. Recent work on LLM-driven code synthesis for performance kernels has established this pattern as state-of-the-art over rule-, mutation-, and random-search baselines for compute and communication kernels [18, 38, 47]. The strategy-enumerate variant is a two-stage layout:

Stage 1: K-way strategy enumeration. One LLM call enumerates K structurally distinct strategy sketches $\{\sigma_1, \ldots, \sigma_K\}$ (we use $K = 5$ throughout the paper) given:

1. the problem's signature $\text{sig}_P : \mathcal{X} \to \mathcal{Y}$;
2. the $\mathcal{S}_P$ reference implementations and their simulator scores;
3. the cost-model parameters Θ flattened into a per-op table.

The LLM emits sketches as $(\text{name}_k, \text{description}_k)$ pairs; a regex parser extracts them. Each σ_k is then implemented by an independent second LLM call into runnable Python code $s_k = \textsc{Impl}(\sigma_k)$, sandbox-executed for correctness against $\text{ref}(\cdot)$ at world sizes $\text{ws} \in \{4, 8\}$ in 32-bit floating-point precision, and benchmarked through $\text{Sim}(\cdot)$. We retain $\{s_k\}$ that pass both correctness and bf16 robustness; up to $K = 5$ candidates survive.

Stage 2: bounded refinement of the top-2. We sort survivors by $\text{Sim}(\cdot)$, take the top two, and refine each in $R = \lceil \text{maxrounds}/2 \rceil$ rounds (we use $R = 2$ throughout the paper). Each refinement round identifies the dominant cost term $\tau^\star \leftarrow \arg\max_{\tau \in \{T_{\text{local}}, T_{\text{coll}}, T_{\text{launch}}, T_{\text{NEFF}}, T_{\text{net}}\}} \tau(s)$, constructs a refine prompt with $(s, \text{Sim}(s), \tau^\star, \text{breakdown}(s), \text{history})$, and emits a mutated s'. If $\text{Sim}(s') < \text{Sim}(s)$ and s' passes the ref+bf16 sandbox, we update $s \leftarrow s'$ and continue.

Section 4.1 compares strategy-enumerate against alternative search styles (a single-trajectory ReAct agent and a multi-island GA variant) on cost and end-to-end outcome.

Cost calculus vs. end-to-end candidate trials. Table 3 quantifies the cost advantage of the loop over the end-to-end-candidate-trial approach (the first existing option discussed in §1). The structural win is that hardware-validation cost is $O(1)$ per problem regardless of how many candidate strategies the LLM explores in Phase 3, whereas the e2e approach pays $O(K)$ cluster hours for K candidates per problem.

	E2E trials	**Our loop**
Score one candidate	full training run (~1 h)	sim. query (~ms)
HW runs / problem	every candidate	sim. winner only
Phase-1 calibration	N/A	~30 min cluster
LLM cost / problem	0	~$2.40
Scaling in K	$O(K)$ cluster h	$O(1)$ cluster h
8 problems, K=5 per problem, 7-node trn1.32xlarge:		
Cluster time	~40 h	~8 h
Cluster cost ($150/h)	~$6,000	~$1,200+$19 LLM

Table 3. Cost calculus, 8-problem suite. The ~5× saving at K=5 is a *lower bound*: strategy-enumerate also scores simulator-refined variants of each candidate for free, so the effective number of strategies the loop explores exceeds K while the loop's cluster cost stays $O(1)$ in the candidate count.

3 Evaluation

Setup (shared). Seven `trn1.32xlarge` instances in a single AWS VPC with EFA (Elastic Fabric Adapter) networking: 32 NeuronCores per node, 224 ranks total, 8 EFA adapters per node at 12.5 GB/s each for cross-node bandwidth. All measurements use `NEURON_NUM_RECENT_MODELS_TO_KEEP=1`, the practitioner default for large-LLM Trainium training: each `mark_step` compiles a fresh HLO graph and only the single most-recently-used NEFF stays resident on device, bounding HBM scratchpad and DMA-ring allocations while surfacing any NEFF-cache-pressure effects on candidate strategies. LLM: Claude Sonnet 4.5 (Phase-3 mutation oracle). Baseline: the AWS-internal production strategy for large-MoE and dense-transformer training (Section 2).

3.1 Setup, methodology, and headline results

Table 4 lists our three harnesses — a per-problem microbench, an OLMoE-10B end-to-end run, and a Llama-7B end-to-end run.

Per-call timings across scopes. Table 5 measures each collective at three progressively wider scopes: **1-node bench** (20-iteration isolated microbench on NeuronLink at 32 ranks), **7-node bench** (same but at 224 ranks with EFA mediating cross-node traffic), and **7-node training** (per-step cost read from inside a real training step, via the elapsed time of the call in its autograd function). The 1-node bench matches what practitioner microbenchmarks exercise; the 7-node training column is the ground truth.

Headline result. Table 6: 1.40× end-to-end on OLMoE-10B and 3.24× on Llama-7B, with algorithm-equivalent descent (real OpenWebText for OLMoE; random-token for Llama-7B, bit-identical per-microbatch-normalized loss). Figure 2 shows the per-call vs. per-step disagreement that drives it. The reported speedup is *steady-state*: OLMoE-10B's wall column (183.6 vs 130.2 min at 2500 steps) is already dominated by steady steps and reflects the 1.40× ratio directly. On Llama-7B at 300 steps the wall column reverses (2.5 vs 5.9 min): OverlayCCL's bundled-graph path pays a one-time compile overhead of ∼3.7 min more than the per-microbatch baseline (fewer, larger NEFF compilations). The steady step-time ratio is 71.3⟶22.0 ms, so OverlayCCL recoups this compile deficit at a rate of 49.3 ms per step and breaks even at ≈4400 training steps (about 5 min of baseline compute). Real training runs are far longer than this break-even; the short 300-step harness undersells the end-to-end effect on wall time, which is why we report steady step time as the headline.

3.2 OLMoE end-to-end training

The OLMoE harness matches OLMoE [35]: multi-head attention with RoPE [55] and RMSNorm [66], MoE block with SwiGLU experts [51], and expert-choice routing [69] (each expert picks a fixed number of tokens, so per-rank send/receive counts are deterministic and need no host-side bookkeeping). Two collectives are exercised end-to-end with independent baseline/OverlayCCL toggles: the AllToAllV around the MoE block (twice per layer) and the distributed cross-entropy over the rank-sharded vocabulary. The *OLMoE-10B* deployment totals ∼11.5 B parameters across 224 ranks. On a 2500-step real-OpenWebText run (Table 6) both backends descend from $\log V \approx 10.4$ to ≈ 6.9, agreeing to within ±0.15 at every 50-step checkpoint. Table 7 sweeps sequence length, model dimension, and layer count; the speedup stays in a tight 1.39–1.41× band, so the headline is configuration-robust rather than shape-tuned.

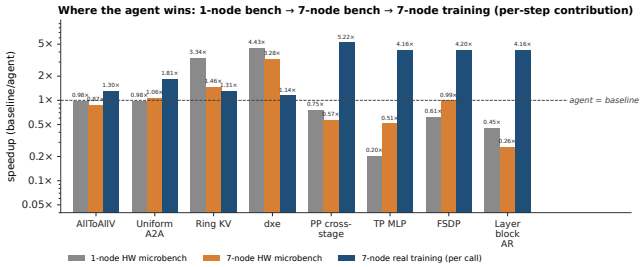

Figure 2. Cross-scope inversion. Per-problem speedup (baseline/agent) at 1-node bench, 7-node bench, and 7-node training scope; bars are strategy-enumerate numbers from Table 5, dashed line marks agent=baseline. Most primitives' per-call ranking at bench scope inverts at training scope.

3.3 Llama training

Dense transformer training introduces a different structural pattern from MoE: each microbatch's collectives sit in their own compiled graph, and the Neuron runtime cannot fuse the collectives of microbatch i with those of microbatch j, so a step with M microbatches issues M separate calls per collective. The four Llama-side problems in Table 1 (PP cross-stage send/recv, TP MLP AR, FSDP weight prefetch, layer-block AR) are all issued in this per-microbatch form by the production AWS recipe [3, 4] and the NeuronX-Distributed-Training samples. Practitioners prefer it because (i) a naive single mega-graph that unrolls the whole forward+backward across all M microbatches exceeds Trainium's memory/compile-time budget while one graph per microbatch fits, and (ii) each microbatch's small isolated collective looks cheap under the per-call microbench practitioners calibrate on. The bundled alternative — which we make precise below and which is distinct from the naive mega-graph — only wins at training scope, where the per-graph-launch overhead paid M times per step by the per-microbatch form starts to dominate.

Concretely, the bundled agent *keeps* the per-microbatch compute graph intact (forward and backward still compile

	Per-problem microbench	OLMoE-10B end-to-end	Llama-7B end-to-end
Scale, steps	1 node · 32 ranks, 5000	7 nodes · 224 ranks, 2500	7 nodes · 224 ranks, 300
Model shell	DeepSeek-MoE-Lite-shaped	OLMoE-architectural-style	Llama-2-7B
LAYERS/DM/HEADS	12 / 2048 / 16	8 / 2048 / 16	32 / 4096 / 32
NEXP/TOPK/EXDIM	64 / 6 / 1408	224 (=ws) / 8 / 1024	—
SEQLEN/BSZ/VOCAB	256 / 1 / 32768	256 / 1 / 32256 (sharded)	1024 / 1 / 32000 (sharded)
Routing	token-choice top-K	expert-choice (exp=1.0, cap=10)	dense
Collective under test	one per script	AllToAllV + dxe	PP + TP MLP + FSDP + layer-block AR

Table 4. Setup of the three evaluation harnesses, bf16 with full autograd backward.

Problem	1-node bench (ms/call)		7-node bench (ms/call)		7-node training (ms/step)	
	baseline	OverlayCCL	baseline	OverlayCCL	baseline	OverlayCCL
OLMoE collectives						
AllToAllV*	**1.83**	1.95	**1.89**	3.04	4410	**3352**
Uniform A2A	**46.95**	47.87	55.4	**52.3**	373.2	**206.2**
Ring KV	3.57	**1.07**	2.95	**2.02**	13.49	**10.32**
dxe (dist. CE)	1.64	**0.37**	1.87	**0.57**	16.74	**14.63**
Llama primitives (per-step contribution from amp1 probe)						
PP cross-stage	**1.78**	2.38	**1.34**	2.37	21.36	**4.11**
TP MLP	**1.91**	2.83	**0.64**	1.67	8.04	**1.93**
FSDP prefetch	**1.21**	2.00	1.37	**1.39**	8.52	**2.01**
Layer-block AR	**0.89**	2.94	**1.15**	4.35	9.40	**2.26**

Table 5. Three measurement scopes per primitive: 1-node 32-rank bench, 7-node 224-rank bench, and 7-node per-step contribution at training scope. Bold marks the faster of each (baseline, OverlayCCL) pair. *AllToAllV is a step-dominant primitive on OLMoE (called 2× per layer × 8 layers per step and carries most of the per-step cost), so its training column approximates the full OLMoE step time in Table 6; other rows carry only a fraction of the step and are properly per-step *contributions*.

Harness	Backend	Wall (min)	Steady (ms)	Final loss	Speedup
OLMoE-10B	baseline	183.6	4404.3	6.843	1.00×
	OverlayCCL	130.2	3139.8	6.945	**1.40×**
Llama-7B	baseline	2.5	71.3	4.970	1.00×
	OverlayCCL	5.9	22.0	4.970	**3.24×**

Table 6. End-to-end speedups over the developer baseline (algorithm-equivalent descent; setup in Table 4).

microbatch-by-microbatch) and only merges the collective calls: M per-microbatch AllReduces (or AllGathers) over gradients or activations are replaced by one call over a $M\times$-larger tensor once per step. This changes the number of dispatches from M to 1 but does not enlarge the model's compute-graph tensor footprint, so the mega-graph OOM signals in (i) never fire on the bundled strategy.

Why isn't collective bundling just the obvious hand fix? On other stacks (NCCL/CUDA) dispatch bundling and gradient bucketing are standard, so the natural objection is that our Llama-side speedups reduce to a known optimization the developer baseline happened to omit. Two facts push back on this framing. First, the microbench-based intuition practitioners calibrate on does not tell them the bundled strategy is a win. The $M\times$-larger single-collective tensor produced by bundling enlarges the collective's own NEFF and shifts more work into the compile-cost term of Eq. 1, which shows up as a per-call regression when the strategy is timed in isolation (compare the per-call columns in Table 5: PP send/recv 1.34 → 2.37 ms, TP MLP 0.64 → 1.67 ms). A shape-inference edge case in the trace pipeline further discourages hand exploration of bundling variants that reshape gradient buffers. Both of those signals fire in the microbench; neither binds at training scope, because the launch tax paid M times per step by the per-microbatch form is much larger than the compile-cost delta from a larger single-collective NEFF (which under `NEURON_NUM_RECENT_MODELS_TO_KEEP=1` amortizes over enough steps to become a rounding error). Second, the simulator surfaces exactly this shift by combining a compile-cost term that scales with the largest single-collective tensor (charging the bundled form) against a launch-tax term that scales linearly in M (charging the per-microbatch form). Under Trainium's calibrated constants the launch tax wins by a factor of M, and the ranking Phase 3 acts on flips. In short: the value of the pipeline here is not

inventing bundling but *surfacing* that its known Trainium-side per-call regressions don't bind at training scope — a call microbenchmarks cannot make and hand-derived cost intuition tends to get wrong. We use two harnesses: a Llama-block sweep at four small shapes (amp1–amp4, Table 7) that probes each primitive individually, and a Llama-7B end-to-end run (Table 6) that drives the full PP+TP+FSDP+layer-block AR strategy under the same per-microbatch vs. bundled toggle. At the per-call layer the four primitives show modest wins (~1.04–2×; Table 5, training column), which multiply into 4–6× per-step contribution ratios because OverlayCCL issues one dispatch per step while the baseline issues M.

M scales the speedup, not the ranking. Unlike OLMoE (where compute and comm scale on the same axes and the speedup stays at ~1.4×), the Llama setup decouples them via the per-microbatch `mark_step` boundaries. OverlayCCL collapses the M per-microbatch collectives into one fused AR/AG per step while the baseline pays M latency-plus-bandwidths, so the headline tracks M: ~3.49–3.62× at M=4 (amp1/amp2) and ~5.84–6.48× at M=8 (amp3/amp4) in Table 7. amp2 (S=2048, M=4) is the default configuration referenced by Table 6.

Harness	Config	base (ms/step)	OverlayCCL (ms/step)	Speedup
OLMoE-10B	SEQLEN=128	2696.9	1943.1	1.39×
	SEQLEN=256 (default)	4404.3	3139.8	1.40×
	SEQLEN=512, DM=1024	4286.1	3076.4	1.39×
	LAYERS=4	2245.8	1593.0	1.41×
Llama-block	amp1 (S=1024, M=4)	46.26	12.68	**3.65×**
	amp2 (S=2048, M=4)	60.08	17.14	**3.51×**
	amp3 (S=1024, M=8)	90.62	15.43	**5.87×**
	amp4 (S=2048, M=8)	112.49	17.95	**6.27×**

Table 7. Configuration-knob sweep (ms/step, 1000-step steady mean). OLMoE is configuration-robust at 1.39–1.41×; Llama-block tracks M because the bundled agent collapses M per-microbatch dispatches into one.

Llama-7B end-to-end. Scaling to a Llama-7B shape and running 200 warmup-dropped steps at 224 ranks gives the second block of Table 6: **3.24×** steady-state over the per-microbatch baseline, with bit-identical per-microbatch-normalized loss (4.970 vs 4.970) on random tokens — the algorithm-equivalence proof for the bundled-vs-per_mb swap, since the only quantities that change between backends are graph layout and dispatch count.

3.4 Multi-seed retest

To check the per-primitive numbers of Table 5 and the Llama-block bundling effect are not single-run artefacts, we re-ran each cell at $n \geq 5$ seeds. The five Llama-side primitives are measured via a per-call probe inside the Llama-block harness (per-call median times per-step call count: N_{MB} for the per-microbatch baseline, 1 for OverlayCCL); Ring KV and dxe report per-step time from their per-problem training loops directly. The end-to-end Llama-block row is a full-step retest of the amp3 configuration (N_{MB}=8) that Table 7 exercises at single-seed. This is distinct from Llama-7B end-to-end (Table 6): the amp3 harness runs one transformer layer per PP stage to isolate the bundling ratio at the largest N_{MB} in our sweep, while Llama-7B runs the full 32-layer stack. Multi-seed retesting the Llama-7B end-to-end shape itself is left to future work; the 3.24× headline there is single-seed. We call a row *clean* when the difference of means exceeds twice the standard error of the difference ($2\sigma_{\bar{x}}$).

Primitive	n	baseline (ms/step)	OverlayCCL (ms/step)	Speedup
Llama-side				
PP send/recv	5	13.22 ± 0.14	1.16 ± 0.22	**11.41×**
FSDP prefetch	5	2.03 ± 0.68	0.45 ± 0.14	**4.54×**
TP MLP	5	1.51 ± 0.69	0.37 ± 0.15	**4.02×**
vocab dxe	5	2.92 ± 0.31	0.88 ± 0.10	**3.30×**
Grad AR	5	1.00 ± 0.10	0.79 ± 0.21	**1.27×**
OLMoE-side				
Ring KV	5	10.51 ± 0.19	6.73 ± 0.12	**1.56×**
dxe	5	15.59 ± 0.09	10.47 ± 0.38	**1.49×**
Uniform A2A	5	3319.9 ± 319.6	2358.1 ± 201.8	**1.41×**
Llama-block harness (amp3 shape)				
Llama-block	5	27.97 ± 0.16	3.70 ± 0.28	**7.56×**

Table 8. Multi-seed retest at $n \geq 5$ seeds; every row is a clean $2\sigma_{\bar{x}}$ OverlayCCL win. The Llama-block row is the 1-layer-per-stage amp3 harness (Table 7), *not* Llama-7B end-to-end (Table 6); it isolates the N_{MB}=8 bundling effect at higher fidelity than amp3's single-seed row and is not comparable to the 3.24× Llama-7B headline.

3.5 Case study: AG+RS vs pack-and-gather

To make one representative cross-scope inversion concrete, we work through AllToAllV — the primitive that dispatches variable-length per-rank chunks around the MoE block. Two strategies compete. **AG+RS**, the strategy AWS practitioners write today, pads every rank's data to a fixed size $c_{\max}$ (the largest chunk any rank sends), calls one `all_gather` to materialize the global buffer, applies a `permute+reshape` to group each destination rank's data, and calls one `reduce_scatter` to extract each rank's share — two collective dispatches. **Pack-and-gather**, the agent's proposal, packs the variable-length data into a fixed-size send buffer where rank i's payload starts at offset $i \cdot c_{\max}$; calls a single `all_gather`; and extracts each rank's slot with a metadata-only view+slice (`gathered[:, rank]`), no permute, no second collective.

In a 20-iteration hardware microbenchmark with the NEFF — Trainium's compiled-graph binary blob — pre-warmed in the compile cache, AG+RS wins per call (1.66 vs 2.75 ms) because the extra dispatch is cheap when the NEFF is cached and its post-collective `reshape+view` is a few microseconds. At training scale three things change. First, each step compiles a fresh HLO (the training-graph IR that

lowers to a NEFF) that varies with the surrounding model graph, and the Neuron runtime evicts older NEFFs under the standard resident-cache cap; AG+RS's two-collective path produces a larger NEFF and more reload events. Second, AG+RS's `permute+reshape` is contiguity-dependent: when the permutation puts a non-leading dim first, PyTorch silently inserts an $O(\text{numel})$ copy of the gathered buffer at strided HBM bandwidth, which on Trainium is ~4× slower than sequential bandwidth (Figure 3). Third, the compilation-cost term scales super-linearly with the largest single-collective tensor, and AG+RS's `all_gather` materializes a tensor $N\times$ larger than the agent's send buffer (where N is world size). None of these show up in the 20-iteration microbenchmark; all show up in end-to-end training.

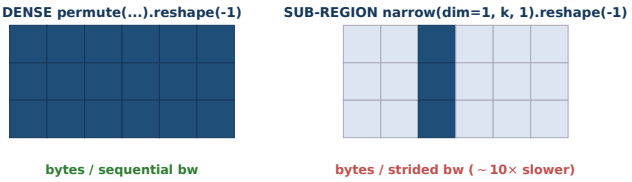

Figure 3. Contiguity-sensitive implicit copies. The same `...reshape(-1)` is free on a dense permuted source (left, sequential bandwidth) but pays an $O(\text{numel})$ strided copy on a sub-region narrow source (right, 4–10× slower).

The simulator's contiguity-aware implicit-copy term charges the strided copy to AG+RS's `permute` chain; its compilation-cost term charges AG+RS for the larger graph. The predicted ranking flips: AG+RS wins the microbench but loses at training scale. Phase 5 ranks by simulator and emits pack-and-gather. End-to-end, this swap alone accounts for ≈25% of the 1.40× OLMoE speedup.

3.6 Cluster-size generalization: 7-node to 3-node

The deployed strategies bake in a few *topology constants* at codegen — total ranks, the half-cluster size for the pipeline-parallel split, and the physical node count — (224, 112, 7) on 7 nodes. To check for overfitting to this shape, we redeploy the same strategies at (96, 48, 3) and (160, 80, 5) with model-shape parameters (MoE expert count, Llama hidden dimension) re-derived so the model fits. If the strategies were 7-node-overfitted, the speedup would disappear or invert at smaller clusters. Table 9 shows it persists in both directions, so the strategies capture device behavior that generalizes across cluster sizes rather than tuning to one topology.

4 Ablation Study

The main results use the default *strategy-enumerate* Phase 3 loop. Two ablations show that (i) alternative Phase 3 styles reach the same performance at ~25% higher search cost, and (ii) stripping the simulator's cost feedback collapses the 1.40× OLMoE headline to 1.00×.

Harness	base (ms)	OverlayCCL (ms)	Speedup
OLMoE-10B 7n (224r)	4404.3	3139.8	1.40×
OLMoE-8B 5n (160r)	3258.0	2173.0	**1.50×**
OLMoE-5B 3n (96r)	2547.4	1422.0	**1.79×**
Llama-7B 7n (224r)	71.30	22.00	3.24×
Llama-7B 5n (160r)	67.94	24.77	**2.74×**
Llama-7B 3n (96r)	67.82	27.20	**2.49×**

Table 9. Cluster-size generalization: same runtimes redeployed at 5-node and 3-node with topology constants refitted. OLMoE speedup widens as cluster shrinks; Llama speedup compresses.

4.1 Alternative Phase-3 loops: cc-react and multi-island GA

cc-react is a single-trajectory ReAct agent on Claude Code [7] maintaining a champion scratchpad plus per-op cost feedback and proposing sequential refinements. **multi-island GA** runs three or more populations seeded from distinct baselines with LLM-mutation per round and occasional champion exchange, modeled on AlphaEvolve [38] with our simulator as fitness. Both share Phase 1 and Phase-4/5's gates.

Result: per-problem cells within ~10% of strategy-enumerate at every scope; OLMoE end-to-end reproduces **1.40×** within < 0.5%; Llama amp1–amp4 within ~10%. All three styles converge to the same deployed Phase-5 winner — Phase 3 search shape does not move the headline.

Why strategy-enumerate is preferred: it reaches the same strategy ~25% faster and at ~20% less LLM token spend (Table 10) under a common 10-call-per-problem budget: one enumeration call for K=5 structurally distinct sketches, K implementation calls, and $2R$=4 refinement calls (two rounds, two candidates) against the simulator. Six of ten calls go to *exploring different ideas*, four to refining the best. The classical greedy-vs-portfolio trade-off [25, 48] applies: one round each on K distinct templates covers more ground than repeated refinement of one, which is why cc-react and multi-island need more rounds to converge.

Phase-3 style	Wall (min)	LLM cost (Sonnet 4.5)
strategy-enumerate (default)	56	**$19**
cc-react	75	$22
multi-island	78	$24

Table 10. Per-style search cost over the eight collective problems on the 7-node cluster.

4.2 No-simulator ablation: necessity of Phase-3 cost feedback

To isolate the simulator's cost feedback, strategy-enumerate runs in `-no-simulator` mode on the same eight problems

under the same budget. Phase 1 still builds the simulator, but its output is hidden from Phase 3; the saved refinement budget lets the agent enumerate candidates, write its own microbenchmarks, and produce code. Phase 5 cannot rank by simulator either, so Phase-4-shape-gate survivors are shown to the LLM with their HW microbench latencies and code and the LLM picks the winner ("LLM-judge" Phase 5). Search wall-time is unchanged (~59 vs ~56 min). With the simulator hidden the LLM-judge falls back to whatever wins the microbench, which for all three load-bearing OLMoE primitives is structurally the inline developer baseline, and the end-to-end ratio collapses to **1.00×** (Table 11).

Harness	base (ms)	no-sim (ms)	Ratio
OLMoE-10B (a2av + dxe)	4399.8	4393.4	**1.00×**
Llama-7B$^{\diamond}$	76.9	76.0	1.01×

Table 11. No-simulator ablation end-to-end. OLMoE **1.00×** is the central finding: stripping simulator scores collapses the 1.40× headline. $^{\diamond}$Llama-7B drops PP, uses TP+FSDP+lbar.

5 Portability to other closed-stack accelerators

The Trainium result raises whether the design transfers to the other closed-stack accelerators the paper names (Google TPU, Meta MTIA) or exploits some Trainium specific. Walking the four phases, every input the pipeline takes from the target platform is either directly replaceable or already platform-agnostic:

- **Phase 1** (calibration) needs to time a single `xm.*`-like primitive at a chosen tensor shape and rank layout. Trainium supplies this via `torch_neuronx+torch_xla`; TPU via `jax.pmap` or `torch_xla/tpu`; MTIA via its internal runtime.
- **Phase 2** (seed pool) needs a small set of correct implementations. Our seeds (§2) — `allgather+slice`, `reduce_scatter+pack`, and `collective_permute-ring` compositions — are expressible in JAX and the MTIA runtime, which expose the same underlying primitives.
- **Phase 3** (LLM proposer) needs no platform-specific input beyond the calibrated simulator and the seed pool.
- **Phase 4** (HW validation) needs to launch a strategy on the target device and read back a step time. Trainium uses `torchrun`; TPU's analogue is `xla_multiprocessing.spawn` or JAX's multi-controller entry.

Nothing in the loop hardcodes Trainium bandwidth constants, NeuronLink topology, or EFA latency: those are Phase-1 measurements re-taken on each deployment. Porting to TPU or MTIA replaces Phase 1's device-timing wrapper and Phase 4's launcher; the LLM proposer and simulator layers are unchanged.

Cost-model constant swap. To defend this more concretely we substitute the six calibrated Trainium constants with published TPU v5p equivalents (ICI 600 GB/s cross-chip, DCN 25 GB/s per port, XLA compile overhead 350 μs, etc.) and rescore the seed pool. A template tied to Trainium topology would move; across the nine templates the Spearman rank correlation is ρ=1.00: the top-3 under Trainium constants (`fused_alltoall`, `hierarchical`, `multinode_hierarchical`) remain the top-3 under TPU-like constants, and the bottom-3 (`allgather_reduce_scatter`, `allgather_slice`, `hybrid_ag_perm`) remain the bottom-3. Absolute scores drop 40–55% for the top-3 (tracking ~3× higher ICI bandwidth), but the ranking Phase 3 acts on is unchanged. The structural design is portable; only calibration is platform-specific. Actual TPU cluster runs are future work.

The design is out-of-scope for NCCL-supported GPU clusters: NCCL (with MSCCL [12], TACCL [50], AutoCCL [64]) already exposes the schedule-level IR our (C2) constraint assumes away, so schedule synthesis — a stronger technique — applies there. Our search targets the regime with no schedule-level IR.

6 Related Work

Distributed training systems above the collective layer. Systems such as ZeRO [42, 45], DeepSpeed [44], Megatron-LM [26, 37, 54], PyTorch DDP and FSDP [30, 67], GPipe [19], PipeDream [36], Horovod [49], BytePS [24], and FlexFlow [22] optimize a different layer than this paper does: they decide *how the training job itself is laid out across devices* — how model weights, optimizer states, activations, and microbatches are partitioned, replicated, or pipelined — and then leave the collective calls themselves to the underlying runtime (NVIDIA's collective-communication library NCCL [39] or AMD's equivalent RCCL on open stacks, or to a schedule-synthesis framework like MSCCL [12] or TACCL [50] where that is available). Our search instead fixes the model-parallel layout and optimizes the next layer down: *how each individual collective is implemented* — which vendor primitive to call, in what order, and how to reshape, pad, or slice the input/output tensors around each call — against the closed Trainium API where `xm.*` primitives cannot be modified or introspected. Strategy-layer systems do not catch vendor-specific pathologies on this stack (e.g., `xm.all_to_all` fails to compile across multiple nodes on Trainium; a tensor reshape that looks free on paper actually forces the runtime to physically copy the full tensor at slower, non-sequential memory bandwidth), and schedule-synthesis frameworks rely on rewriting schedules *below* the

runtime boundary, which Neuron disallows. We treat `xm.*` as fixed and search strategies *above* it.

Cost-model simulators for distributed training. SimAI [61], ASTRA-sim [43], and ASTRA-sim 2.0 [63] score training-system choices against a cost-model simulator instead of running them end-to-end. They are written and tuned against a fixed reference configuration; we discuss the resulting drift problem and our LLM-recalibrated alternative in §1 and §2.2.

Collective synthesis below an open boundary. A line of work synthesizes collective algorithms by exposing the in-fabric schedule: SCCL [8] casts it as a constraint problem; MSCCL/MSCCLang [12] compiles a DSL to an NCCL schedule IR; TACCL [50] adds communication sketches and topology-aware search; Blink [60] targets multi-GPU servers; CoCoNet [20] jointly optimizes compute+communication; AutoCCL [64] auto-tunes low-level NCCL parameters under compute-comm interference. All depend on a vendor library that allows replacing or parameterizing the in-fabric schedule, a prerequisite met on open NVIDIA/AMD stacks but not on closed-stack accelerators such as AWS Trainium (Neuron Runtime), Google TPU (where collectives are emitted by the XLA compiler as nodes in its compute-graph IR, called HLO — High-Level Operations), and Meta MTIA — where the in-fabric schedule lives behind a vendor compiler and is not externally addressable. The Neuron stack we use as our case study has no analogue of MSCCL-IR; we therefore search above the primitives, not beneath them. The same closed-stack constraint holds on TPU and MTIA, which makes the strategy-search paradigm we describe potentially applicable to them as well (§5).

Algorithm discovery with LLMs. FunSearch [47] pioneered the LLM-as-mutator loop for mathematical discovery; AlphaEvolve [38] extended it to code-level optimization; ShinkaEvolve [27] emphasized sample-efficient open-ended evolution. AlphaTensor [15] and AlphaDev [34] use deep RL for matmul/sorting discovery. LLM code-generation work (Codex [9], AlphaCode [31], Self-Refine [33], Reflexion [53], ToT [65], Self-Instruct [62]) further established LLMs as practical mutation oracles given a feedback loop. All search the *algorithm* or *program text* space; in our setting the same strategy admits many syntactic forms and the cost surface is dominated by hardware quirks (NEFF cache, EFA pipelining, implicit copies) none of these frameworks model. We take the LLM-mutation loop structure from this lineage and graft it onto a closed-stack-appropriate cost function; the headline configuration is a single-trajectory ReAct agent because in our setting the cost model and profiling toolkit already do the work island parallelism would otherwise do — and the single-trajectory variant is cheaper. Multi-island appears as an ablation.

Kernel autotuning and graph optimization. TVM [10], AutoTVM [11], Ansor [68] search kernel-implementation choices; Halide [40] and Triton [56] provide DSLs; Tensor Comprehensions [59], MLIR [28], TASO [21] operate at the graph level. LLM-driven kernel-optimization systems (STARK [5], K-Search [6], Astra [13], KernelEvolve [46]) refine individual GPU kernels by 1.25–17×. Our problem is one level above: the Neuron compiler writes each `xm.*` kernel; we vary which primitives are composed and how, not their internal schedule.

MoE and Llama communication patterns. The Sparsely-Gated MoE layer [52], GShard [29], Switch [16], ST-MoE [70], DeepSpeed-MoE [41], Mixtral [23], DeepSeek-MoE [14], and OLMoE [35] together established that MoE expert dispatch dominates the training step once active parameters exceed a few hundred million. Expert-choice routing [69] fixes per-expert receive counts and removes per-step host-side count computation from the AllToAllV path; we use it in the end-to-end OLMoE evaluation. On Llama [57, 58], the PP cross-stage send/recv + TP MLP + FSDP weight-prefetch strategy we evaluate matches the standard training recipe. Trainium practitioners typically issue one collective per microbatch (e.g. one TP-MLP all-reduce per microbatch for a training step of M microbatches uses M all-reduce calls). We show that, when the per-microbatch payloads can share an underlying buffer, the same logical strategy can also be implemented by issuing a single collective per training step that covers all M microbatches at once — trading M launch costs for one. Whether this trade is profitable depends on the device (a larger payload may take longer to compile or exceed memory limits), so we let the search make that choice per problem rather than fixing it by hand.

7 Conclusion

We presented a search loop that finds faster collective-communication strategies on top of a closed-source vendor library, where two structural properties make optimization hard: (C1) no visibility into the runtime's behavior, and (C2) no low-level control over how primitives are scheduled. For (C1), we have an LLM agent build a cost-model simulator on the target deployment itself, calibrating its internal numbers against the actual device and model shape — so the simulator stays workload-faithful. For (C2), we accept the vendor API as fixed and search one layer above it, over which primitives to call, in what order, and with what tensor pre- and post-processing. On AWS Trainium the deployed strategies deliver 1.40× end-to-end on OLMoE-10B (with algorithm-equivalent descent on real OpenWebText) and 3.24× on Llama-7B over an internal-AWS-optimized production baseline. An ablation that hides the simulator from the LLM collapses the OLMoE speedup back to 1.00×, confirming that the on-device-calibrated simulator — not the LLM —

is what picks the strategy that actually wins at real training runs.

References

[1] Amazon Web Services. 2024. AWS Neuron Documentation: Trainium Architecture. https://awsdocs-neuron.readthedocs-hosted.com/en/latest/about-neuron/arch/neuron-hardware/trainium.html. Accessed 2026-05-15.

[2] Amazon Web Services. 2024. AWS Trainium2 and Trn2 Instances for Generative AI. AWS Product Documentation. https://aws.amazon.com/ai/machine-learning/trainium/

[3] Amazon Web Services. 2024. Training with Tensor Parallelism (NeuronX Distributed). AWS Neuron SDK Documentation. https://awsdocs-neuron.readthedocs-hosted.com/en/latest/libraries/neuronx-distributed/tutorials/training.html

[4] Amazon Web Services. 2024. Tutorials for NeuronX Distributed. AWS Neuron SDK Documentation. https://awsdocs-neuron.readthedocs-hosted.com/en/latest/libraries/neuronx-distributed/tutorials/index.html

[5] Anonymous. 2025. STARK: Strategic Team of Agents for Refining Kernels. https://arxiv.org/abs/2510.16996 arXiv:2510.16996.

[6] Anonymous. 2026. K-Search: LLM Kernel Generation via Co-Evolving Intrinsic World Model. https://arxiv.org/abs/2602.19128 arXiv:2602.19128.

[7] Anthropic. 2025. Claude Code: An interactive CLI agent for code. https://www.anthropic.com/claude-code

[8] Zixian Cai, Zhengyang Liu, Saeed Maleki, Madanlal Musuvathi, Todd Mytkowicz, Jacob Nelson, and Olli Saarikivi. 2021. Synthesizing Optimal Collective Algorithms. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*. ACM, 62–75. doi:10.1145/3437801.3441620

[9] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021).

[10] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Meghan Cowan, Haichen Shen, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.

[11] Tianqi Chen, Lianmin Zheng, Eddie Yan, Ziheng Jiang, Thierry Moreau, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. Learning to Optimize Tensor Programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 31. https://arxiv.org/abs/1805.08166

[12] Meghan Cowan, Saeed Maleki, Madanlal Musuvathi, Olli Saarikivi, and Yifan Xiong. 2023. MSCCLang: Microsoft Collective Communication Language. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 502–514. doi:10.1145/3575693.3575724 Extends MSCCL; arXiv:2201.11840.

[13] Stanford CS. 2025. Astra: A Multi-Agent System for GPU Kernel Performance Optimization. https://arxiv.org/abs/2509.07506 arXiv:2509.07506.

[14] Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y. K. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. 2024. DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. https://arxiv.org/abs/2401.06066

[15] Alhussein Fawzi, Matej Balog, Aja Huang, Thomas Hubert, Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, et al. 2022. Discovering Faster Matrix Multiplication Algorithms with Reinforcement Learning. *Nature* 610 (2022), 47–53.

[16] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *Journal of Machine Learning Research* 23, 120 (2022), 1–39.

[17] Aaron Gokaslan, Vanya Cohen, Ellie Pavlick, and Stefanie Tellex. 2019. OpenWebText Corpus. http://Skylion007.github.io/OpenWebTextCorpus.

[18] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence. *arXiv preprint arXiv:2401.14196* (2024).

[19] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. 2019. GPipe: Efficient Training of Giant Neural Networks Using Pipeline Parallelism. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[20] Abhinav Jangda, Jun Huang, Guodong Liu, Amir Hossein Nodehi Sabet, Saeed Maleki, Youshan Miao, Madanlal Musuvathi, Todd Mytkowicz, and Olli Saarikivi. 2022. Breaking the Computation and Communication Abstraction Barrier in Distributed Machine Learning Workloads. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 402–416. doi:10.1145/3503222.3507778

[21] Zhihao Jia, Oded Padon, James Thomas, Todd Warszawski, Matei Zaharia, and Alex Aiken. 2019. TASO: Optimizing Deep Learning Computation with Automatic Generation of Graph Substitutions. In *ACM Symposium on Operating Systems Principles (SOSP)*.

[22] Zhihao Jia, Matei Zaharia, and Alex Aiken. 2019. Beyond Data and Model Parallelism for Deep Neural Networks. In *Proceedings of Machine Learning and Systems (MLSys)*.

[23] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, et al. 2024. Mixtral of Experts. *arXiv preprint arXiv:2401.04088* (2024).

[24] Yimin Jiang, Yibo Zhu, Chang Lan, Bairen Yi, Yong Cui, and Chuanxiong Guo. 2020. A Unified Architecture for Accelerating Distributed DNN Training in Heterogeneous GPU/CPU Clusters. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.

[25] Levente Kocsis and Csaba Szepesvári. 2006. Bandit Based Monte-Carlo Planning. In *Machine Learning: ECML 2006*. Springer, 282–293.

[26] Vijay Anand Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. 2023. Reducing Activation Recomputation in Large Transformer Models. In *Proceedings of Machine Learning and Systems (MLSys)*.

[27] Robert Tjarko Lange et al. 2025. ShinkaEvolve: Towards Open-Ended and Sample-Efficient Program Evolution. *arXiv preprint arXiv:2509.19349* (2025). https://arxiv.org/abs/2509.19349

[28] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. 2021. MLIR: Scaling Compiler Infrastructure for Domain Specific Computation. In *IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*.

[29] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2021. GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding. In *International Conference on Learning Representations (ICLR)*.

[30] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, and Soumith Chintala. 2020. PyTorch Distributed: Experiences on Accelerating Data Parallel Training. *Proceedings of the VLDB Endowment* 13, 12 (2020).

[31] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, et al. 2022. Competition-Level Code Generation with AlphaCode. *Science* 378, 6624 (2022), 1092–1097.

[32] Hao Liu, Matei Zaharia, and Pieter Abbeel. 2024. Ring Attention with Blockwise Transformers for Near-Infinite Context. In *International Conference on Learning Representations (ICLR)*.

[33] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[34] Daniel J. Mankowitz, Andrea Michi, Anton Zhernov, Marco Gelmi, Marco Selvi, Cosmin Paduraru, Edouard Leurent, et al. 2023. Faster Sorting Algorithms Discovered Using Deep Reinforcement Learning. *Nature* 618 (2023), 257–263.

[35] Niklas Muennighoff, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Jacob Morrison, Sewon Min, Weijia Shi, Pete Walsh, Oyvind Tafjord, Nathan Lambert, Yuling Gu, Shane Arora, Akshita Bhagia, Dustin Schwenk, David Wadden, Alexander Wettig, Binyuan Hui, Tim Dettmers, Douwe Kiela, Ali Farhadi, Noah A. Smith, Pang Wei Koh, Amanpreet Singh, and Hannaneh Hajishirzi. 2025. OLMoE: Open Mixture-of-Experts Language Models. In *International Conference on Learning Representations (ICLR)*. arXiv:2409.02060.

[36] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R. Devanur, Gregory R. Ganger, Phillip B. Gibbons, and Matei Zaharia. 2019. PipeDream: Generalized Pipeline Parallelism for DNN Training. In *ACM Symposium on Operating Systems Principles (SOSP)*.

[37] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. 2021. Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*.

[38] Alexander Novikov et al. 2025. AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery. *arXiv preprint arXiv:2506.13131* (2025). https://arxiv.org/abs/2506.13131

[39] NVIDIA Corporation. 2024. NCCL: NVIDIA Collective Communications Library. https://github.com/NVIDIA/nccl. Accessed 2026-05-15.

[40] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. 2013. Halide: A Language and Compiler for Optimizing Parallelism, Locality, and Recomputation in Image Processing Pipelines. In *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*.

[41] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. 2022. DeepSpeed-MoE: Advancing Mixture-of-Experts Inference and Training to Power Next-Generation AI Scale. In *International Conference on Machine Learning (ICML)*.

[42] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*.

[43] Saeed Rashidi, Srinivas Sridharan, Sudarshan Srinivasan, and Tushar Krishna. 2020. ASTRA-SIM: Enabling SW/HW Co-design Exploration for Distributed DL Training Platforms. In *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*.

[44] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*.

[45] Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shuangyan Yang, Minjia Zhang, Dong Li, and Yuxiong He.

2021. ZeRO-Offload: Democratizing Billion-Scale Model Training. In *USENIX Annual Technical Conference (ATC)*.

[46] Meta Research. 2025. LLM-Guided Evolutionary Kernel Optimization: From Research to Production Kernels. https://mlai.blog/2025-12-20-llm-kernel-optimization

[47] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. 2024. Mathematical Discoveries from Program Search with Large Language Models. *Nature* 625, 7995 (2024), 468–475. doi:10.1038/s41586-023-06924-6

[48] Stuart Russell and Peter Norvig. 2010. *Artificial Intelligence: A Modern Approach* (3 ed.). Prentice Hall.

[49] Alexander Sergeev and Mike Del Balso. 2018. Horovod: Fast and Easy Distributed Deep Learning in TensorFlow. *arXiv preprint arXiv:1802.05799* (2018).

[50] Aashaka Shah, Vijay Chidambaram, Meghan Cowan, Saeed Maleki, Madan Musuvathi, Todd Mytkowicz, Jacob Nelson, Olli Saarikivi, and Rachee Singh. 2023. TACCL: Guiding Collective Algorithm Synthesis using Communication Sketches. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, 593–612. https://www.usenix.org/conference/nsdi23/presentation/shah

[51] Noam Shazeer. 2020. GLU Variants Improve Transformer. arXiv:2002.05202.

[52] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. In *International Conference on Learning Representations (ICLR)*.

[53] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[54] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. *arXiv preprint arXiv:1909.08053* (2019).

[55] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. 2024. RoFormer: Enhanced Transformer with Rotary Position Embedding. *Neurocomputing* (2024).

[56] Philippe Tillet, H. T. Kung, and David Cox. 2019. Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*. ACM, 10–19. doi:10.1145/3315508.3329973

[57] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, et al. 2023. LLaMA: Open and Efficient Foundation Language Models. *arXiv preprint arXiv:2302.13971* (2023).

[58] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, et al. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv preprint arXiv:2307.09288* (2023).

[59] Nicolas Vasilache, Oleksandr Zinenko, Theodoros Theodoridis, Priya Goyal, Zachary DeVito, William S. Moses, Sven Verdoolaege, Andrew Adams, and Albert Cohen. 2018. Tensor Comprehensions: Framework-Agnostic High-Performance Machine Learning Abstractions. *arXiv preprint arXiv:1802.04730* (2018).

[60] Guanhua Wang, Shivaram Venkataraman, Amar Phanishayee, Nikhil Devanur, Jorgen Thelin, and Ion Stoica. 2020. Blink: Fast and Generic Collectives for Distributed ML. In *Proceedings of Machine Learning and Systems (MLSys)*, Vol. 2. 172–186.

[61] Xiaoyang Wang et al. 2025. SimAI: Unifying Architecture Design and Performance Tuning for Large-Scale Large Language Model Training with Scalability and Precision. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI '25)*.

[62] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. 2023. Self-Instruct: Aligning Language Models with Self-Generated Instructions. In *Annual Meeting of the Association for Computational Linguistics (ACL)*.

[63] William Won, Jaehong Heo, Saeed Rashidi, Srinivas Sridharan, Sudarshan Srinivasan, and Tushar Krishna. 2023. ASTRA-sim2.0: Modeling Hierarchical Networks and Disaggregated Systems for Large-Model Training at Scale. In *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*.

[64] Guanbin Xu, Zhihao Le, Yinhe Chen, Zhiqi Lin, Zewen Jin, Youshan Miao, and Cheng Li. 2025. AutoCCL: Automated Collective Communication Tuning for Accelerating Distributed and Parallel DNN Training. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. USENIX Association, 667–683. https://www.usenix.org/conference/nsdi25/presentation/xu-guanbin

[65] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[66] Biao Zhang and Rico Sennrich. 2019. Root Mean Square Layer Normalization. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[67] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, Alban Desmaison, Can Balioglu, et al. 2023. PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel. *Proceedings of the VLDB Endowment* 16, 12 (2023).

[68] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, Joseph E. Gonzalez, and Ion Stoica. 2020. Ansor: Generating High-Performance Tensor Programs for Deep Learning. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.

[69] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew M. Dai, Zhifeng Chen, Quoc V. Le, and James Laudon. 2022. Mixture-of-Experts with Expert Choice Routing. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 35. https://arxiv.org/abs/2202.09368

[70] Barret Zoph, Irwan Bello, Sameer Kumar, Nan Du, Yanping Huang, Jeff Dean, Noam Shazeer, and William Fedus. 2024. ST-MoE: Designing Stable and Transferable Sparse Expert Models. In *International Conference on Learning Representations (ICLR)*. arXiv:2202.08906.